

Product Requirements Document (PRD)

Project Title: Intelligent PA (AI-Powered Personal Booking Agent) **Domain:** LLM Agents / LangGraph / Model Context Protocol (MCP) / Conversational AI

1. Problem Statement Summary

The objective is to design and develop an AI-powered personal booking and scheduling agent capable of interacting with users in natural language to manage meetings, appointments, and general calendar events. The system must understand intent, maintain conversational context, resolve scheduling conflicts, and summarize bookings using a structured, agentic workflow.

2. Product Vision & Positioning

We are not building a simple conversational wrapper for a calendar API. The product is an **Intelligent Personal Assistant (PA)**. It differentiates between personal and professional contexts, proactively manages schedule health, and relies on deterministic, graph-based routing rather than standard prompt-and-pray LLM execution.

3. Core Features & "Above and Beyond" Differentiators

Baseline Requirements

- **Natural Language Intent Parsing:** Extract time, participants, and context from messy human inputs.
- **Calendar Management:** Check availability, create, update, and delete events.
- **Automated Invites:** Automatically dispatch Gmail invites and attach Google Meet links via the calendar provider.

The "Winning" Differentiators

- **Human-in-the-Loop (HITL) Conflict Resolution:** The PA will never autonomously overwrite existing high-priority meetings. Conflicts pause the execution graph and surface a structured "Approve/Reject/Reschedule" widget to the user UI.
- **Cross-Session Memory:** A persistent state layer that remembers user preferences (e.g., "no meetings before 10 AM") across completely different chat sessions.
- **Pre-Meeting Intelligence (Multi-Tool RAG):** Before booking, the PA utilizes a search/RAG tool to pull a quick summary of the meeting topic or participant and injects it into the calendar event description.
- **Voice Input:** Integration of a fast Speech-to-Text (STT) model (like Whisper) on the frontend for frictionless scheduling.
- **Proactive Background Operations:** System monitoring to detect schedule density and suggest buffer times or lunch breaks.

4. System Architecture & Tech Stack

To ensure a robust, production-like environment, the architecture is decoupled into a clear frontend, a deterministic graph backend, and standardized tool protocols. *Note: High-Level Design (HLD) and Low-Level Design (LLD) diagrams should be generated prior to coding to visualize the state flow.*

- **Orchestration Framework:** LangGraph (Python) for state-machine logic and multi-agent routing.
- **Backend Application:** FastAPI to expose the LangGraph execution endpoints and handle HITL webhooks.
- **Frontend UI:** Streamlit or a basic React app to handle chat interface, voice recording, and HITL approval widgets.
- **Tool Integration Protocol:** Model Context Protocol (MCP).
 - Instead of raw Google Calendar API REST calls, the system will operate as an MCP Client connecting to an open-source Google Calendar MCP Server (e.g., [guinacio/mcp-google-calendar](#) or [deciduus/calendar-mcp](#)).
- **Memory / State Persistence:** SQLite or standard LangGraph checkpointing for state management between graph nodes.

5. Multi-Agent Workflow (The State Machine)

The LangGraph implementation will utilize a multi-agent separation of concerns rather than a single massive system prompt.

1. **Router Node:** Analyzes the initial query to determine the required path (e.g., Schedule, Cancel, Query, or General Chat).
2. **Extractor Node:** Parses the natural language into a structured JSON payload (Time, Date, Attendees, Intent).
3. **MCP Execution Node:** Interfaces directly with the MCP server to check availability or mutate calendar state.
4. **Negotiator / HITL Node:** Triggered if the MCP Node returns a conflict. Generates alternative slots, pauses graph execution, sends options to the UI, and awaits user selection.
5. **Self-Healing / Reflection Node:** Catches formatting errors or missing parameter errors from the MCP server, prompts the LLM to correct its own payload, and re-attempts the action without crashing the application.

6. Edge Cases & Robustness Guidelines

- **Temporal Grounding:** Every state payload must dynamically include the exact current datetime and the user's localized timezone. The LLM must be explicitly prompted to calculate relative time (e.g., "next Tuesday at 3 PM") against this grounded variable.
- **Full Event Lifecycle (CRUD):** The agent must be equally capable of reading, updating, and explicitly deleting events via semantic search of the user's schedule, not just creating new ones.
- **Entity Resolution Assumption:** For the scope of the prototype, the system assumes exact email addresses will be provided in the natural language prompt for meeting attendees.